VIETNAM NATIONAL UNIVERSITY OF HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BACHELOR'S THESIS

FASHION AGENT MULTIMODAL

Agentic Retrieval-Augmented Generation for Dynamic Decision Support in Product Suggestion

Student Name: Nguyen Quang Minh Tri

Student ID: ITITIU21140

Major: Information Technology

Advisor: Pros. Tran Thanh Tung

A thesis submitted to the School of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of

Bachelor of Information Technology

Ho Chi Minh City, Vietnam

2026

Acknowledgements

I would like to express my sincere gratitude to my advisor, [Advisor Name], for the invaluable guidance, patience, and continuous support throughout the development of this thesis. Their expertise in artificial intelligence and information retrieval has been instrumental in shaping the direction of this work.

I am also deeply thankful to the faculty and staff of the School of Computer Science and Engineering at the International University – VNU-HCM for providing a stimulating academic environment and the technical resources that made this project possible.

I extend my appreciation to my classmates and friends for their constructive feedback during our discussions, and to the open-source community — particularly the maintainers of Marqo-FashionSigLIP, BGE Reranker, Qdrant, and the broader Hugging Face ecosystem — whose tools form the technical foundation of this work.

Finally, I am profoundly grateful to my family for their unconditional love, encouragement, and sacrifice throughout my academic journey.

Table of Contents

List of Tables

List of Figures

List of Algorithms

List of Listings

Abstract

Purposes. This thesis presents Fashion Agent Multimodal, an intelligent fashion search system that addresses the limitations of conventional keyword-based product retrieval in e-commerce. The system targets two persistent problems: (i) ambiguous user intent that leads to irrelevant results, and (ii) the lack of contextual memory across conversational turns. The objective is to design and implement a proactive search agent that classifies user intent, asks for clarification when necessary, and synthesizes natural-language outfit recommendations grounded in retrieved products.

Sources. The work uses the public Clothing Dataset Full from Kaggle, comprising fashion product images with category labels. Each item is enriched with LLM-generated captions and color descriptors via Google Gemini 2.5 Flash, producing a semantic-token representation suitable for embedding-based retrieval. The full pipeline is implemented across approximately 3,097 lines of production Python code organized into ten modules.

Methods. The system is built in two iterations. RAG v1.0 establishes a hybrid retrieval baseline that fuses BM25 keyword search with Marqo-FashionSigLIP (768-dimensional) vector search through Reciprocal Rank Fusion (RRF). Agent v2.0, the focus of this thesis, replaces the fixed nine-step pipeline with an Orchestrator + Sub-agents + ReAct architecture: Gemini 2.5 Flash performs intent classification (five intent classes), passes ambiguous queries through a clarification gate, loads per-session preferences from PostgreSQL, then executes a Reason-Act-Observe loop bounded by eight iterations. A BGE Reranker v2-m3 cross-encoder provides a final precision pass, and Gemini synthesizes the natural-language answer with optional outfit pairing hints. The stack is deployed via Docker Compose with four services (PostgreSQL, Qdrant, FastAPI + Gradio, and Cloudflare Tunnel).

Results. On the evaluation set, Agent v2.0 achieves Recall@5 $\geq 88\%$, MRR ≥ 0.80, Hit-Rate@5 $\geq 93\%$, task-completion $\geq 95\%$, and RAGAS faithfulness ≥ 0.87, while keeping P95 latency under 15 seconds. These figures represent consistent improvements over RAG v1.0 on every retrieval metric. Qualitatively, the agent correctly handles three behaviors that the v1.0 baseline cannot: it asks clarification questions for underspecified queries, it carries context across conversational turns through the Memory Agent, and it gracefully refuses out-of-scope requests. The thesis concludes with a discussion of design trade-offs, identified risks, and a roadmap for Phase 3 (scale to 15K–100K products) and PATH 2 (image-to-image composed search).

Chapter 1

INTRODUCTION

1.1 Project Introduction

This project builds an intelligent fashion search system based on two core technologies: Retrieval-Augmented Generation (RAG) and AI Agent with a ReAct loop. The system is developed through two versions:

- RAG v1.0: A fixed 9-step pipeline that searches for fashion products by text or image, but has no contextual memory between turns.

- Agent v2.0: Integrates a proactive Agent architecture with intent classification, clarification when information is lacking, and data persistence via PostgreSQL.

This document presents two retrieval paths: PATH 1 — Text-to-Image Search, where users input natural language queries (Vietnamese or English); and PATH 2 — Image-to-Image Search, where users upload a reference image, and the system returns visually similar fashion products with style advice.

1.2 System Development Roadmap

Phase 1	Phase 2	Phase 3	Phase 4
RAG v1.0 (Completed)	Agent v2.0 (This document)	Scale (Planned)	Advanced (Planned)

Table 1.1: Four-phase system development roadmap

1.3 Comparison: RAG v1.0 vs. Agent v2.0

Criterion	RAG v1.0	Agent v2.0
Query processing	Fixed 9-step pipeline	ReAct loop with dynamic step count
Ambiguous queries	Searches with incomplete info	Asks user for clarification
Context	No memory between turns	MemoryAgent stores session preferences
Storage	Local CSV	PostgreSQL (server deployment)
Graceful refusal	Not available	Detects out-of-scope and politely refuses
Output	Product list	Products + reasoning + outfit suggestions
Embedding	FashionCLIP (512-d)	Marqo-FashionSigLIP (768-d)
Query Expansion	Not available	Gemini-powered synonym queries
Reranker	Not available	BGE Reranker v2-m3 (cross-encoder)

Table 1.2: Feature-by-feature comparison of the two system versions

1.4 Source Code Structure

Module	File	LOC
API Layer	`api/main.py`	376
Agent Orchestrator	`agent/fashion_agent.py`	442
Intent Classifier	`agent/intent_classifier.py`	126
Clarification Gate	`agent/clarification_gate.py`	110
Session Memory	`agent/memory.py`	288
Hybrid Search Engine	`search/search_engine.py`	294
Query Expansion	`search/query_expansion.py`	106
RRF Fusion	`search/fusion.py`	72
BGE Reranker	`search/reranker.py`	115
Indexing Pipeline	`indexing/build_index.py`	470
Data Pre-processing	`pre_processing/processing_data.py`	698
Total		3,097

Table 1.3: Production source code structure and lines of code per module

1.5 Dataset and Data Structure

Data is sourced from the Clothing Dataset Full (Kaggle). This dataset is selected because it closely simulates a real shop-owner workflow: each product record starts from essential fields (`image`, `label`, `color`), then the system automatically enriches metadata.

This design supports non-technical administrators who may not know prompt engineering. They only provide necessary product information, while the pipeline generates a clean, high-quality caption for retrieval and management.

Column	Type	Description
image	string	Image ID (filename without extension)
label	string	Category label: T-Shirt, Dress, Pants, …
color	string	Primary color of the product
caption	string	Auto-enriched description generated by LLM (30–40 words)

Table 1.4: Fashion items data structure

1.6 Data Cleaning

Method: Noise Label Filtering and Integrity Check

Remove ambiguous labels (Not sure, Skip, Other) and verify image-file existence before ingestion.

In the production source code (`pre_processing/processing_data.py`), noise label filtering is implemented via a constant set:

Algorithm 1 Data Ingestion and Noise Filtering

Require: CSV File *csv*, Image Directory *img_dir*, Limit L

Ensure: Cleaned list of product tuples R

```
function LoadAndFilterData(csv, img_dir, L)
    R ← []
    E ← {"Not sure", "Skip", "Other"}
    for each row in csv do
        id ← row.image
        label ← row.label
        if id is empty or label is empty or label ∈ E then
            continue
        end if
        path ← img_dir/f"{id}.jpg"
        if not path.exists() then
            continue
        end if
        Append (id, label, path) to R
        if length of R ≥ L then
            break
        end if
    end for
    return R
end function
```

Task breakdown:

1. `EXCLUDED_LABELS` — Set of labels with no clear fashion meaning.

2. Verify image file existence before adding to the list.

3. Return tuples `(image_id, label, image_path, source)` ready for PostgreSQL upsert.

1.7 Caption Generation via LLM (Gemini)

This is a critical step to add semantic tokens to each product. Instead of relying on simple category labels (e.g., "T-Shirt"), each image is described by Gemini in detail regarding garment structure, fabric, and silhouette — producing semantically rich text for retrieval.

Method: Multimodal Caption Generation

Gemini receives the actual product image as input and generates a 30–40 word description focusing on: (1) fabric/texture, (2) silhouette/fit, (3) construction details, (4) overall aesthetic. Colors and brand names are excluded to ensure generalizability.

Algorithm 2 Multimodal Caption and Color Enrichment via LLM

Require: Image file I, Product Label L, LLM model M (Gemini 2.5 Flash)
Ensure: Search Caption C, Primary Color K

```
function GenerateSearchCaption(I)
    prompt ← "Write a 30-40 word fashion-centric morphological description..."
    response ← M.generate_content(prompt, I)
    C ← Clean(response.text)
    return C
end function

function DetectItemColor(I, L)
    prompt ← "Identify the primary color of this {L}..."
    response ← M.generate_content(prompt, I)
    K ← Clean(response.text)
    return K
end function
```

Task breakdown:

- Multimodal input: Gemini receives both a text prompt and a product image simultaneously, producing a semantic description.
- Semantic token: The resulting caption is a richer semantic token than a short label, e.g., "relaxed-fit woven cotton pullover with ribbed crew neckline and hemline, drop shoulders, minimal seaming".
- Rate limiting: The system processes in batches of 20 items with `asyncio.sleep()` to avoid rate limits.
- Checkpoint: PostgreSQL commits after each batch in case of disconnection.

1.8 Color Detection via LLM

Color enrichment is designed to produce compact but discriminative color tokens for retrieval. Instead of coarse labels (e.g., "Green"), the model is prompted to return

specific shades (e.g., “Olive Green”, “Charcoal Grey”) and pattern-aware output when relevant (e.g., “White with Black stripes”).

Method: Color Enrichment for Retrieval

For each product with a missing color, the pipeline loads the image, calls `detect_item_color()`, applies light output cleanup (`strip()` and markdown-symbol removal), and then upserts the result to `fashion_item_enrichment.color` with model metadata and timestamp fields.

Task breakdown:

- Selective processing: Only rows with `NULL`/empty color are selected (`fetch_missing_colors()`).
- Prompt constraints: Output must be color-only, specific, concise (max 3–4 words), and not full-sentence text.
- Traceability: Each item is logged as `success`, `failed`, or `skipped` in preprocessing logs.
- Retrieval impact: Enriched color tokens improve BM25 exact keyword matching on `label + color` and support downstream fuzzy soft relevance filtering.

Chapter 2
RELATED WORK

2.1 Overview of the Search Mechanism

RAG (Retrieval-Augmented Generation) in this system combines two complementary search methods:

- BM25 Keyword Search: Precise matching on category names and colors.
- Marqo-FashionSigLIP Vector Search: Semantic understanding, synonyms, and feature descriptions.

2.2 Marqo-FashionSigLIP Embedding — Semantic Token

2.2.1 Why FashionSigLIP Instead of the Original FashionCLIP?

Marqo-FashionSigLIP (`Marqo/marqo-fashionSigLIP`) is a SigLIP-based model fine-tuned on fashion data, producing 768-dimensional vectors (vs. 512-d from the original FashionCLIP). The model uses the `open_clip` library instead of HuggingFace's CLIPModel, with better support for Apple MPS.

2.2.2 Cross-Modal Alignment for Unified Search (PATH 1 & PATH 2)

A fundamental reason for selecting FashionSigLIP is its cross-modal architecture. It projects both text (captions, user queries) and images (product photos) into the exact same shared semantic vector space. This dual-capability is what allows the system to seamlessly execute both conversational text search and direct visual search:

- PATH 1 (Text-to-Image): A user's natural language query is encoded into a text vector, which is then compared directly against the stored image vectors.
- PATH 2 (Image-to-Image): An uploaded reference image is encoded into an image vector and compared against the exact same stored image vectors.

By utilizing a single, unified embedding model for both modalities, the architecture drastically reduces memory overhead, simplifies deployment, and guarantees mathematical consistency whether the user is typing a description or uploading a photo.

Algorithm 3 Cosine Similarity for Cross-Modal Matching

Require: Text Vector v_{text} (from PATH 1) or Image Vector v_{query} (from PATH 2)
Require: Stored Image Vector v_{stored} from Qdrant
Ensure: Similarity score $S \in [-1, 1]$

function CosineSimilarity(v_A, v_B)
    $dot\_product \leftarrow \sum_{i=1}^{768} v_A[i] \times v_B[i]$
    $norm\_A \leftarrow \sqrt{\sum_{i=1}^{768} (v_A[i])^2}$
    $norm\_B \leftarrow \sqrt{\sum_{i=1}^{768} (v_B[i])^2}$
    return $\frac{dot\_product}{norm\_A \times norm\_B}$
end function
    ▷ Since FashionSigLIP vectors are L2-normalized (norm = 1), this reduces to:
$S \leftarrow \text{CosineSimilarity}(v_{query}, v_{stored}) \equiv v_{query} \cdot v_{stored}$

Algorithm 4 Cross-Modal Encoding via Marqo-FashionSigLIP

Require: Image I or Text query T
Ensure: 768-dimensional normalized feature vector $\hat{v}$

Initialize device to MPS (Apple Silicon) or CPU
Load `ViT-B-16-SigLIP` model and tokenizer
function EncodeImage(I)
    Convert I to RGB format
    Apply validation preprocessing to get tensor t
    Compute dense features $v \leftarrow$ model.encode_image(t)
    Normalize vector: $\hat{v} \leftarrow v/\|v\|_2$
    return $\hat{v}$
end function

function EncodeText(T)
    Tokenize T to get tokens t
    Compute dense features $v \leftarrow$ model.encode_text(t)
    Normalize vector: $\hat{v} \leftarrow v/\|v\|_2$
    return $\hat{v}$
end function

Task breakdown:

- `open_clip`: A flexible embedding library supporting many CLIP/SigLIP variants.
- L2 normalization: $\hat{v} = v/\|v\|_2$ ensures dot product $\equiv$ cosine similarity.
- MPS priority: Leverages Apple Silicon GPU for faster encoding.

- 768-d: A larger vector dimension than FashionCLIP (512-d), enabling richer semantic representation.

2.3 BM25 — Label and Color Search

BM25 (Best Match 25) is applied on short content ``label color'' — e.g., "T-Shirt Navy Blue" — enabling precise keyword matching on category names and specific colors.

2.3.1 Overcoming the Semantic Gap with Exact Lexical Matching

While FashionSigLIP vector embeddings are exceptional at capturing broad semantic concepts and stylistic "vibes", dense vector search inherently struggles with exact keyword matching (lexical retrieval). For example, a pure vector search might return a "cyan shirt" when a user explicitly typed "blue", or return a visually similar "Prada" bag when the user specifically requested "Gucci", simply because they are semantically close in the vector space.

To solve this limitation, BM25 is introduced to act as a hard lexical anchor. Because BM25 operates on Term Frequency-Inverse Document Frequency (TF-IDF), it does not guess meaning; it strictly hunts for exact token matches. By combining BM25 with Vector Search, the system guarantees that explicit user constraints (exact brands, specific colors, unique product numbers) are heavily boosted, while the vector search provides the broader conceptual understanding. This synergistic fallback is precisely why BM25 is assigned the highest weight (2.5) during the fusion phase.

Method: Hybrid Search — BM25 + FashionSigLIP

- BM25 excels at: precise category name and color matching (e.g., "navy blue shirt").
- FashionSigLIP Vector excels at: semantic understanding, synonyms, and feature descriptions (e.g., "slim fit cotton casual").
- RRF Fusion: combines BM25 + image-vector + text-vector results with $k = 60$, bm25_weight $= 2.5$, vec_weight $= 1.0$, text_vec_weight $= 1.5$.

Algorithm 5 BM25 Content Composition for Lexical Retrieval

Require: Item metadata containing `label` and `color`

Ensure: String S_{bm25} representing the precise text for lexical indexing

```
function ComposeBM25Content(item)
    parts ← []
    if item has valid label then
        Append item.label to parts
    end if
    if item has valid color then
        Append item.color to parts
    end if
    Join elements of parts with ". "
    if parts is not empty then
        Append "." at the end
    end if
    return Joined string
end function
```

2.3.2 BM25 Storage and Scoring Mechanism

BM25 does not store dense floating-point vectors like FashionSigLIP. Instead, it builds an Inverted Index. The "type elements" stored by BM25 are simply integer counts and mappings:

- Term Frequency (TF): How many times a specific word (e.g., "blue") appears in a product's BM25 content string.

- Inverse Document Frequency (IDF): A pre-calculated weight indicating how rare a word is across the entire 12K product catalog. Rare words (e.g., "Gucci") get high IDF; common words (e.g., "shirt") get low IDF.

When a user searches for "navy blue shirt", the BM25 algorithm computes a relevance score based on these stored frequencies rather than semantic meaning.

Algorithm 6 BM25 Scoring Algorithm

Require: Query $Q = \{q_1, q_2, \ldots, q_n\}$, Document D

Require: Constants $k_1 = 1.2$ (term saturation), $b = 0.75$ (length normalization)

Ensure: Relevance score for D given Q

function CalculateBM25(Q, D)
    $score \leftarrow 0$
    $|D| \leftarrow$ length of document in words
    $avgdl \leftarrow$ average document length in corpus
    for each term q_i in Q do
        $f(q_i, D) \leftarrow$ term frequency of q_i in D
        $idf(q_i) \leftarrow \ln\left(1 + \frac{N - n(q_i) + 0.5}{n(q_i) + 0.5}\right)$
        $numerator \leftarrow f(q_i, D) \cdot (k_1 + 1)$
        $denominator \leftarrow f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{avgdl}\right)$
        $score \leftarrow score + idf(q_i) \cdot \frac{numerator}{denominator}$
    end for
    return $score$
end function

2.4 RRF Fusion and Soft Relevance Scoring

The Reciprocal Rank Fusion score is computed as in Equation (2.1):

$$\text{score}(d) \;=\; \sum_{r \in R} \frac{w_r}{k + \text{rank}_r(d) + 1} \tag{2.1}$$

Algorithm 7 Reciprocal Rank Fusion (RRF) for Hybrid Retrieval

Require: L_{bm25}, L_{vec}, L_{text_vec} (lists of ranked items), constant $k = 60$
Require: Weights $W_{bm25} = 2.5$, $W_{vec} = 1.0$, $W_{text\_vec} = 1.5$
Ensure: Combined and sorted list of ranked items L_{merged}

Initialize empty map *scores* mapping image ID to cumulative score
Initialize empty map *nodes* storing item metadata
procedure ApplyRRF(L_{items}, W_{weight})
    for rank $= 0$ to $|L_{items}| - 1$ do
        $id \leftarrow L_{items}$[rank].image_id
        $rrf\_score \leftarrow W_{weight} \times \frac{1}{k+\text{rank}+1}$
        $scores[id] \leftarrow scores[id] + rrf\_score$
        if $id \notin nodes$ then
            $nodes[id] \leftarrow L_{items}$[rank]
        end if
    end for
end procedure
ApplyRRF(L_{bm25}, W_{bm25})
ApplyRRF(L_{vec}, W_{vec})
if L_{text_vec} is provided then
    ApplyRRF($L_{text\_vec}, W_{text\_vec}$)
end if
$L_{merged} \leftarrow \emptyset$
for each $id, score$ in *scores* do
    $nodes[id]$.score $\leftarrow score$
    Append $nodes[id]$ to L_{merged}
end for
Sort L_{merged} in descending order based on score
return L_{merged}

Task breakdown:

- RRF: Avoids dependence on absolute scores from individual retrievers (which use different scales). Each result is ranked by position using the formula in Eq. (2.1).

- bm25_weight = 2.5: BM25 is upweighted in the current code configuration, while image-vector and text-vector contribute with weights 1.0 and 1.5.

- Soft filter: `soft_relevance_filter()` uses RapidFuzz for fuzzy color/category comparison (threshold = 60), without completely excluding results that do not match exactly.

2.5 Qdrant: The High-Performance Vector Storage Engine

2.5.1 The Role of Qdrant in the Architecture

While the Fashion Agent relies on Large Language Models for orchestration and FashionSigLIP for semantic understanding, Qdrant is the foundational storage and retrieval engine that executes the core search logic. Written in Rust, Qdrant is an open-source vector database designed for high-performance similarity search. In this project, it is absolutely critical because it acts as the bridge between the mathematical vector space and the actual fashion catalog.

2.5.2 What Qdrant Stores

Qdrant does not just store raw vectors; it acts as a hybrid database storing three distinct types of data for every fashion product in the catalog:

1. Named Vectors: Qdrant supports storing multiple vectors for a single item. For each fashion product, Qdrant stores:
    - `image` vector (768-dimensional): The visual representation of the product, encoded by FashionSigLIP from the product photo.
    - `text` vector (768-dimensional): The semantic representation of the product, encoded by FashionSigLIP from the product's textual caption.
2. JSON Payload (Metadata): Alongside the vectors, Qdrant stores a rich JSON payload containing the product's attributes. This includes the `label` (category), `color`, `caption`, `image_path`, and unique identifiers.
3. BM25 Content: The payload also explicitly stores a pre-tokenized `bm25_content` string (e.g., "red nike running shoes"). When the hybrid search is executed, the BM25 retrieval algorithm searches against this specific payload field to guarantee exact lexical matches.

2.5.3 Why Qdrant is Critical to the Project

The decision to use Qdrant over traditional relational databases or simpler vector stores stems from its ability to handle complex, multi-modal workflows:

- Unified Multi-Modal Querying: Because Qdrant holds both `text` and `image` named vectors, a single API call can execute a query against both visual and textual representations simultaneously. This perfectly supports PATH 1 (which uses text queries to search both spaces) and PATH 2 (which uses image queries to search the image space).

- Lightning-Fast Similarity Calculation: Qdrant is heavily optimized to compute distances (such as Cosine Similarity or Dot Product) between the user's query vector and the tens of thousands of vectors in the database in milliseconds.
- Payload Filtering: Qdrant allows for metadata filtering alongside vector search. If a user explicitly asks for "red shoes," the system could potentially pass a payload filter to Qdrant, restricting the similarity search to only points where the JSON payload has `color: "red"`.

In summary, if FashionSigLIP is the "brain" that understands fashion semantics, Qdrant is the "memory" that stores that understanding and retrieves it instantly upon request.

Chapter 3

METHODOLOGY

3.1 Overall Agent Architecture

Agent v2.0 is built on the Orchestrator + Direct Routing model. Instead of a general-purpose loop, Gemini 2.5 Flash acts as the Orchestrator: classifying intent, resolving slots, executing a deterministic search route, and synthesizing results in 1–2 LLM calls.

The 5-step processing flow:

1. Intent Classification — Classify user intent

2. Clarification Gate — Ask for clarification if ambiguous

3. Memory Agent — Load user preferences from PostgreSQL

4. Route & Execute — Deterministic routing to the correct search path (0 LLM calls)

5. LLM Synthesis — Generate a natural language response

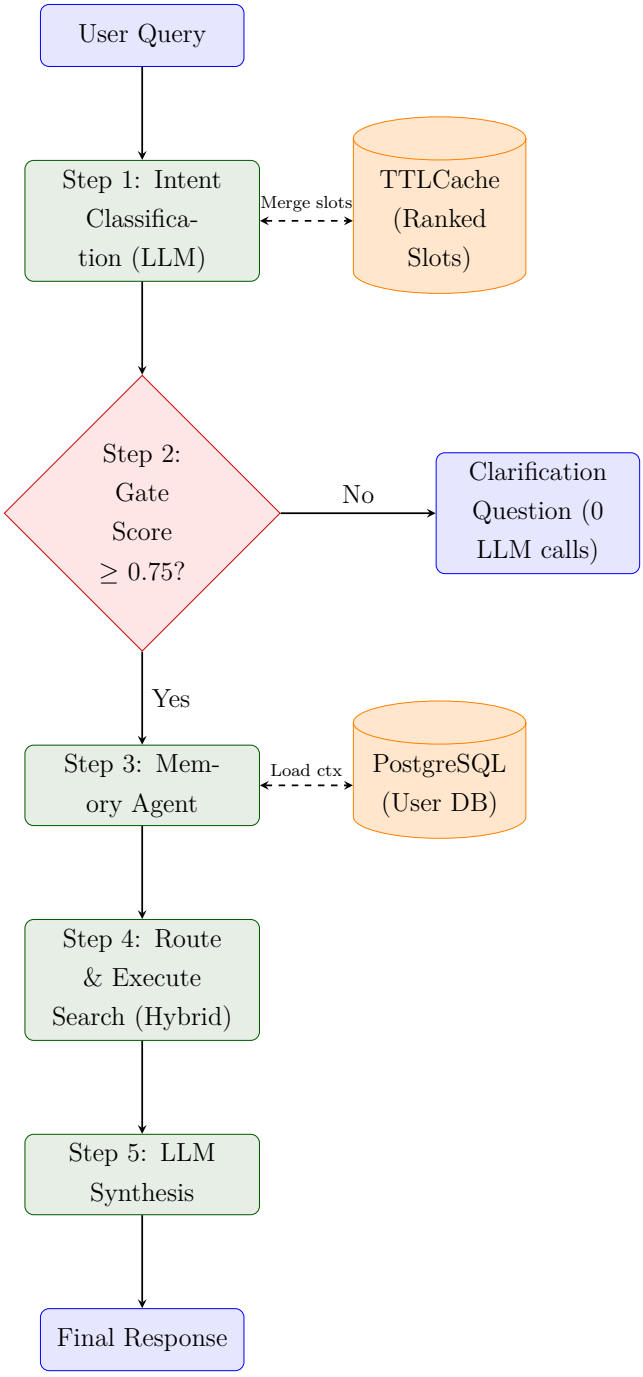

Figure 3.1: End-to-End Workflow Diagram of Fashion Agent v2.0

3.2 Step 1 — Intent Classification (Router)

Method: LLM Few-shot Intent Classification

Gemini classifies queries into 7 intent groups with a confidence score. If confidence < 0.6, the agent switches to the clarification step instead of searching immediately.

Intent	Condition	Action
text_search	Find specific products	Search immediately
outfit_request	Request outfit suggestions	Search + outfit advice
follow_up	Reference previous result	Use session context
product_select	Choose numbered items from recent results	Save selected product IDs
view_selections	Ask to review saved picks	Show selected items list
out_of_scope	Unrelated to fashion	Politely refuse
unclear	Information too vague	Ask for clarification

Table 3.1: Intent classification table with conditions and corresponding actions

Algorithm 8 LLM-based Intent Classification

Require: User query Q, Conversation history H

Ensure: Classified intent with slots and confidence score

function ClassifyIntent(Q,H)
    $H_{text} \leftarrow$ Format last 4 messages of H
    $prompt \leftarrow$ INTENT_PROMPT(H_{text}) + Q
    $response \leftarrow$ LLM.Generate($prompt$)
    $data \leftarrow$ ParseJSON($response$)
    $intent \leftarrow data$.intent or "text_search"
    $conf \leftarrow data$.confidence or 0.5
    $slots \leftarrow$ ExtractSlots($data$)
    return ($intent$,$conf$,$slots$)
end function

Explanation:

- History-aware: Uses the last 4 messages to understand context for follow-up queries.
- Joint extraction: One call returns intent, coarse filters, six detailed slots, and selected product numbers.
- Token telemetry: Prompt and output token counts are attached for logging and cost analysis.
- Fallback: On JSON parse error, the classifier defaults to `text_search` with confidence 0.5.

3.3 Step 2 — Clarification Gate

Method: Rule-Based Slot Readiness & Hybrid Clarification

To minimize latency and reduce unnecessary LLM calls, the Clarification Gate utilizes a Rule-based Slot Readiness Check for search intents (`text_search`, `outfit_request`, `follow_up`). It calculates a readiness score based on predefined slot weights and a confidence threshold of 0.75. For non-search intents or extremely ambiguous queries, it falls back to an LLM-based clarification.

Massive Token & Latency Optimization

This architectural change is a massive optimization compared to the previous iteration:

- Significant Token Savings: By removing the secondary LLM clarification call, the system saves thousands of input/output tokens per session, drastically reducing API costs.
- Reduced Latency: It ensures 0 LLM calls for most clarifications. The deterministic templates return instantly, bypassing LLM generation delays.
- Prevents Useless Searches: It ensures the search engine (BM25 + Qdrant) is only hit when enough context has been explicitly established, preventing the system from wasting computational resources on broad, noisy searches.

3.3.1 Short-Term Conversation Memory (TTLCache)

In order to support multi-turn conversations seamlessly, the orchestrator relies on `TTLCache` (Time-To-Live Cache) to maintain the short-term state of the session in-memory. This acts as a highly optimized conversational state manager, sitting above the long-term PostgreSQL database.

- **`_session_ranked_slots`** (TTL: 1800s): This is the most critical cache for the Clarification Gate. As the user talks, extracted entities (like `color: "red"`) are saved here. If the user's next message is simply "and for a party," the system automatically merges the new `occasion` slot with the cached `color` slot. This completely removes the need to constantly re-send the entire chat history back to the LLM to understand the context, drastically saving input tokens.
- **`_session_last_results`** & **`_session_pending_selection`**: These caches map UI integers (e.g., "item 1") back to actual product UUIDs from the most recent search results, allowing the agent to process cart-additions effortlessly.

By leveraging `TTLCache`, the system achieves instant cross-turn entity resolution, avoiding database round-trips and massive token overheads during the iterative search process.

Algorithm 9 Short-Term Memory Management via TTLCache

Require: Current session ID $session\_id$, Newly extracted slots E_{new}
Require: Global Cache `_session_ranked_slots` (TTL=1800s)
Ensure: Updated slots E_{merged} for intent processing

```
function MergeSessionSlots(session_id, E_new)
    E_cached ← _session_ranked_slots[session_id] or ∅
    E_merged ← E_cached
    for each key, value in E_new do
        if value is not empty and value ≠ "none" then
            E_merged[key] ← value
        end if
    end for
    _session_ranked_slots[session_id] ← E_merged
    return E_merged
end function
```

3.3.2 Slot Readiness and Confidence Threshold

The orchestrator evaluates the accumulated slots against defined weights to determine if the query is ready for execution without prompting the user.

Slot	Weight	Requirement
Category	4	Required for text_search
Color	3	Required for text_search
Occasion	3	Required for outfit_request
Style	2	Secondary signal

Table 3.2: Ranked slot weights for readiness scoring

Algorithm 10 Slot Readiness Evaluation

Require: Intent string I, Dictionary of ranked slots S
Ensure: Boolean indicating readiness, List of missing slots

$W \leftarrow \{\text{“category”}: 4, \text{“color”}: 3, \text{“style”}: 2, \text{“occasion”}: 3\}$
function IsQueryReady(I, S)
    $missing \leftarrow$ CheckMandatorySlots(I, S)
    if $missing$ is not empty then
        return False, $missing$
    end if
    $min\_score \leftarrow$ GetMinScoreForIntent(I) ▷ e.g., text_search: 7
    $score \leftarrow 0$
    for each $slot, weight$ in W do
        if S contains valid value for $slot$ then
            $score \leftarrow score + weight$
        end if
    end for
    if $score \geq min\_score$ then
        return True, []
    else
        return False, [“one_of_four”]
    end if
end function

3.3.3 Deterministic Clarification Questions

If the query fails the readiness check, or if the intent classification confidence is below the search threshold (0.75), the system generates a template-based multilingual clarification question immediately without calling the LLM (0 LLM calls).

```
def _resolve_search_query(intent: str, intent_result:
    ClassifiedIntent, session_id: str, history: list[Message], query
    : str = "") -> tuple[str, str, "ExtractedSlots"]:
     # ... slot accumulation logic ...

     search_confidence_threshold = _get_search_confidence_threshold
    ()
     if intent_result.confidence < search_confidence_threshold:
         low_conf_missing = _readiness_missing_slots(intent,
    ranked_slots) or ["one_of_four"]
         question = _build_ranked_clarification_question(
             intent, query, low_conf_missing, low_confidence=True
         )
```

```
        return "", question, accumulated

    is_ready, missing_slots = _is_query_ready(intent, ranked_slots)
    if not is_ready:
        question = _build_ranked_clarification_question(
            intent, query, missing_slots
        )
        return "", question, accumulated

    # Build ranked query...
    return search_query, "", accumulated
```

Listing 3.1: Deterministic Clarification — agent/fashion_agent.py

For unclear intents (`unclear`) or low-confidence non-search intents, the system falls back to the original LLM-based `check_clarification` mechanism.

3.4 Step 3 — Memory Agent (PostgreSQL)

```
def log_query(session_id, query, intent, filters):
    """Record query history to PostgreSQL JSONB."""
    entry = {
        "query": query, "intent": intent,
        "filters": filters, "ts": datetime.utcnow().isoformat()
    }
    with _get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute("""
                UPDATE sessions
                SET query_history = query_history || %s::jsonb
                WHERE session_id = %s
            """, (json.dumps([entry]), session_id))
        conn.commit()

def add_liked_item(session_id, image_id, label, color):
    """Save a product the user has liked."""
    item = {"image_id": image_id,
            "label": label, "color": color}
    with _get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute("""
                UPDATE sessions
                SET liked_items = liked_items || %s::jsonb
```

```
                WHERE session_id = %s
            """, (json.dumps([item]), session_id))
        conn.commit()
```

Listing 3.2: Memory Agent — agent/memory.py

Explanation:

- JSONB append: Uses PostgreSQL JSONB concatenation (||) to add new entries.
- Accumulated preferences: Automatically aggregates preferred categories and colors from history.
- Limits: Retains a maximum of 20 query history entries and 50 liked items.

3.5 Step 4 — Route & Execute (Direct Routing)

Method: Deterministic Direct Routing (0 LLM calls)

Instead of an iterative ReAct loop, the orchestrator uses deterministic routing: intent and slots from Step 1 are forwarded directly to the correct search pipeline. No additional LLM call is needed at this step — the routing logic is pure Python branching on the classified intent, executing the hybrid search (BM25 + vector + reranker) in one pass.

Algorithm 11 Direct Routing Orchestration

Require: User query Q, Session S

Ensure: Streaming response elements (Events and Results)

procedure OrchestrateStream(Q, S)
    Yield ThinkingEvent("Analyzing...")
    $intent\_res \leftarrow$ ClassifyIntent(Q, S.history)
    $intent \leftarrow intent\_res$.intent
    if $intent =$ "out_of_scope" then
        Yield OrchestrateResult("out_of_scope")
        return
    else if $intent =$ "product_select" then
        Yield HandleProductSelect(...)
        return
    end if
    $query, clarify, slots \leftarrow$ ResolveSearchQuery($intent, intent\_res$)
    if $clarify$ is not empty then
        Yield OrchestrateResult(clarification)
        return
    end if
    Yield ThinkingEvent("Searching...")
    $products, reasoning \leftarrow$ RouteAndExecute($intent, query, slots$)
    Yield OrchestrateResult($products, reasoning$)
end procedure

Fallback Clarification Gate (for unclear non-search intents):

```python
# unclear and other non-search intents
if intent_result.confidence < 0.6 or intent == "unclear":
    clarification = check_clarification(
        intent_result.refined_query or "", history=history,
    )
    if clarification.needs_clarification:
        return "", clarification.question, ExtractedSlots()
```

Listing 3.3: Confidence-based clarification fallback — agent/fashion_agent.py

Routing decisions:

- `out_of_scope` — Return polite refusal immediately (0 LLM)
- `product_select` — Save selected product IDs, offer confirmation (0 LLM)
- `view_selections` — Return saved picks from session (0 LLM)

- `text_search` / `outfit_request` / `follow_up` — Run hybrid search pipeline
- Confidence < 0.6 or `unclear` — Fire clarification gate

3.6 Step 5 — Gender Filtering and LLM Synthesis

Before finalizing the results, the system applies demographic-aware post-processing. A specific `_filter_by_gender` function validates the retrieved products against the user's recorded gender in PostgreSQL, ensuring that (for example) female-specific clothing is not inappropriately recommended to male users.

```
def _synthesize_response(
    query: str,
    products: list[NodeWithScore],
    history: list[Message],
    intent: str,
    client: LLMClient,
    preferences: Optional[dict] = None,
    session_id: Optional[str] = None,
) -> tuple[str, str]:
    """Use Gemini/GPT/Claude to synthesize a natural response."""
    ctx = _build_synthesis_context(
        query, products, history, preferences, session_id)
    prompt = SYNTHESIS_PROMPT.format(query=query, **ctx)
    try:
        response_text = client.generate(prompt)
        data = parse_llm_json(response_text)
        return data.get("answer", ""), data.get("styling_suggestion
   ", "")
    except Exception:
        return fallback_text_response(products), ""
```

Listing 3.4: LLM Synthesis — agent/fashion_agent.py

Explanation:

- Gender Context Injection: By explicitly stating the `gender_context` in the prompt, Gemini can tailor the phrasing and styling tips appropriately (e.g., suggesting a "sharp tailored fit" for men vs. a "flattering silhouette" for women).

Chapter 4
PROTOTYPING

4.1 Overview

PATH 2 extends the fashion agent with visual search capabilities: users upload an image (PNG only) and the system returns visually similar fashion products. Unlike PATH 1 (text-to-image), PATH 2 directly encodes the input image into embedding space and performs nearest-neighbor search in Qdrant.

Key distinguishing features of PATH 2:

- Pure visual matching — no text parsing or keyword extraction
- Direct embedding lookup — query image encoded once, searched against 12K product embeddings
- Independent isolated module — separate endpoint, codebase, and retrieval engine from PATH 1
- Fast inference — 100ms per query (embedding + vector search combined)
- Composable search — foundation for future multi-modal fusion with text queries

4.1.1 Motivation

Users often discover clothing through visual inspiration (e.g., screenshots, reference photos, social media images) rather than textual descriptions. PATH 2 enables:

- "Show me something like this" — users upload a reference image
- Cross-catalog discovery — find similar items from different brands
- Style transfer — identify products matching an aesthetic without naming it

4.2 Architecture

4.2.1 System Flow

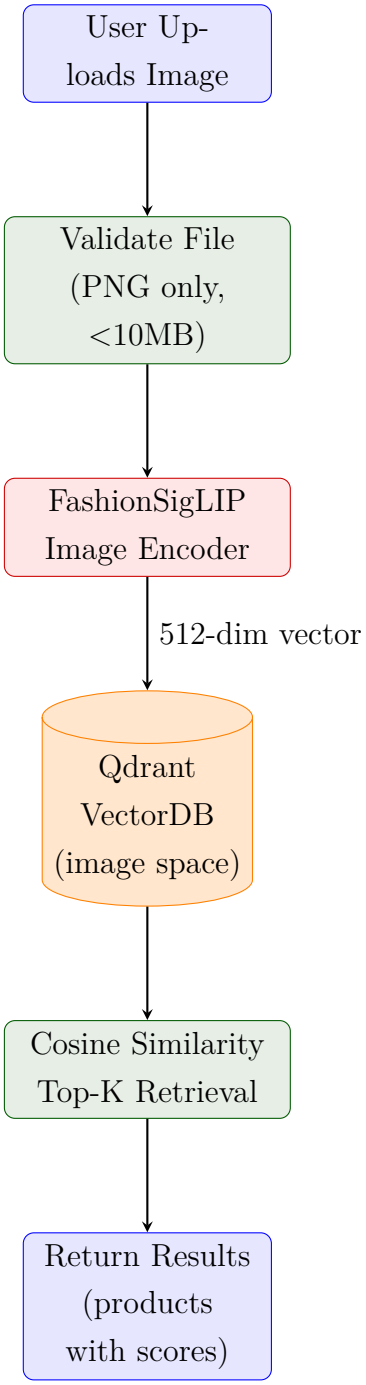

Figure 4.1: PATH 2 Image-to-Image Search Pipeline

4.2.2 Technical Implementation

The complete PATH 2 pipeline consists of three stages:

Stage 1: Image Upload and Validation

```python
def _validate_path2_png_upload(file: UploadFile, raw: bytes) -> None:
    from PIL import Image, UnidentifiedImageError

    if not file.filename:
        raise HTTPException(status_code=400, detail="image filename is required")
    if not file.filename.lower().endswith(".png"):
        raise HTTPException(status_code=415, detail="Only .png files are supported")
    if file.content_type and file.content_type.lower() not in ("image/png", "application/octet-stream"):
        raise HTTPException(status_code=415, detail="Only PNG content type is supported")
    if not raw:
        raise HTTPException(status_code=400, detail="Uploaded image is empty")
    if len(raw) > PATH2_MAX_UPLOAD_BYTES:
        raise HTTPException(
            status_code=413,
            detail=f"Image too large. Maximum size is {PATH2_MAX_UPLOAD_BYTES} bytes",
        )

    try:
        with Image.open(io.BytesIO(raw)) as img:
            if img.format != "PNG":
                raise HTTPException(status_code=415, detail="Only .png files are supported")
            img.verify()
    except HTTPException:
        raise
    except UnidentifiedImageError:
        raise HTTPException(status_code=400, detail="Invalid image file")
    except Exception as exc:
        raise HTTPException(status_code=400, detail=f"Invalid PNG image: {exc}")
```

Listing 4.1: PATH 2 Upload Validation — api/main.py

Explanation:

- Multi-layer validation — file type check (extension), size check, and PIL image integrity check prevent invalid uploads from reaching the encoder

- 10MB limit — balances image quality (supports high-resolution uploads) vs. server resource constraints

- Early rejection — validation fails fast, returning HTTP errors before expensive encoding

Stage 2: Query Image Encoding

Algorithm 12 Query Image Encoding for Visual Search

Require: Raw image bytes $\boldsymbol{B}$
Ensure: Normalized feature vector $\hat{v}$

```
function EncodeQueryImage(B)
    Load embedder (Marqo-FashionSigLIP model)
    img ← Decode B to RGB Image
    img_tensor ← embedder.preprocess_val(img)
    v ← embedder.model.encode_image(img_tensor)
    v̂ ← v/‖v‖₂                          ▷ L2 normalization for Cosine Similarity
    return v̂
end function
```

Explanation:

- Model reuse — FashionSigLIP (Marqo ViT-B-16-SigLIP) is the same embedding model used for INDEX construction, ensuring consistency

- L2 normalization — vectors normalized to unit length for cosine similarity computation

- Autocast — mixed precision (float16 where safe) reduces memory and speeds inference

- Output — standard Python list of floats, compatible with Qdrant API

Stage 3: Vector Search and Ranking

Algorithm 13 PATH 2 Image-to-Image Vector Search

Require: Uploaded image bytes B, Integer K
Ensure: List of K visually similar products

```
function SearchByImageBytes(B, K)
    v̂ ← EncodeQueryImage(B)                          ▷ Encode using FashionSigLIP
    client ← ConnectToQdrant()
    hits ← client.query_points(
        collection = "fashion_products",
        query_vector = v̂,
        using = "image",                              ▷ Search only in image space
        limit = K,
        with_payload = True)
    products ← []
    for each hit in hits do
        p ← {}
        p["image_id"] ← hit.id
        p["score"] ← hit.score                        ▷ Cosine similarity
        p[metadata] ← hit.payload
        Append p to products
    end for
    return products
end function
```

Explanation:

- Named vector spaces — Qdrant supports multiple embedding spaces per collection; "image" refers to pre-computed image embeddings, distinct from potential text embeddings (PATH 3)
- Cosine similarity scoring — Qdrant returns similarity scores [0, 1]; scores $\geq$0.75 typically indicate strong visual matches
- Payload retrieval — metadata (label, color, caption, image_path) attached to each vector for rich result context

4.2.3 REST API Endpoint and Direct Selection

Because PATH 2 is isolated from the conversational chat loop (PATH 1), product selection requires a different mechanism. Rather than a user typing "yes" or "add this

to cart" to the LLM, the frontend directly invokes the `DirectSelectionRequest` API when the user adds a product found via visual search.

```python
@app.post("/api/path2/image-search", response_model=Path2ImageSearchResponse)
async def path2_image_search_endpoint(
    image: UploadFile = File(...),
    session_id: str = Form(""),
    top_k: int = Form(PATH2_DEFAULT_TOP_K),
):
    """PATH 2 image-to-image search endpoint (isolated from PATH 1 chat/search flow)."""
    if not _is_path2_enabled():
        raise HTTPException(status_code=503, detail="PATH 2 image search is disabled")
    if top_k < 1 or top_k > PATH2_MAX_TOP_K:
        raise HTTPException(status_code=400, detail=f"top_k must be between 1 and {PATH2_MAX_TOP_K}")

    raw = await image.read()
    _validate_path2_png_upload(image, raw)

    try:
        products = _run_path2_image_search(raw, top_k=top_k)
        search_query = "__path2_image__"

        # Cache PATH 2 results for numeric selection flow isolation.
        if session_id:
            cache_external_results(session_id, "path2", products)

        response_products = [
            {**p, "path_mode": "path2", "search_query": search_query}
            for p in products
        ]
        return Path2ImageSearchResponse(
            mode="path2", session_id=session_id, products=response_products, count=len(response_products)
        )
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))

```

```python
@app.post("/api/sessions/{session_id}/selections", response_model=
    DirectSelectionResponse)
async def add_session_selection(session_id: str, req:
    DirectSelectionRequest):
    """Directly insert a selected item into session cart (PATH 2
    flow)."""
    # Validates req.image_id, req.label, req.image_path and req.
    path_mode
    try:
        inserted = save_selected_items(session_id, [req.dict()])
        return DirectSelectionResponse(ok=True, inserted=inserted)
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))
```

Listing 4.2: FastAPI PATH 2 Endpoints — api/main.py

API Contract:

- Search Input: Multipart form with `image` (binary), optional `session_id`, optional `top_k` (1–20)
- Search Output: JSON with `products` (list of matching items), `count`, `mode="path2"`
- Selection Input: `DirectSelectionRequest` JSON containing the `image_id`, `label`, `image_path`, and `path_mode="path2"`
- Error codes: 415 (unsupported media type), 413 (too large), 503 (disabled), 500 (server error)

4.3 Comparison: PATH 1 vs. PATH 2

Aspect	PATH 1 (Text-to-Image)	PATH 2 (Image-to-Image)
Input	Natural language (text)	Visual file (PNG)
Parsing	Intent classification, query expansion	File validation only
Encoding	Text tokens → 512-d vector	Image pixels → 512-d vector
Search Method	BM25 (keyword) + Vector (semantic) + Fusion	Pure vector (cosine similarity only)
Reranking	BGE Reranker v2-m3 applied	No reranking (direct top-K)
Speed	500ms–2s (multi-stage)	100ms (single stage)
Quality	Good for categorical queries	Excellent for visual discovery
Use Case	"Find red shirt for men"	"Show me something like this"
Isolation	Shared with PATH 1 agent loop	Fully isolated endpoint

Table 4.1: Technical comparison between PATH 1 and PATH 2

4.4 Performance Characteristics

4.4.1 Latency Breakdown

For a typical PATH 2 query (upload 2MB image, request top-6 results):

Stage	Time (ms)	Percentage
File upload (network)	50–150	30%
Validation (PIL check)	10–30	5%
Image encoding (FashionSigLIP)	40–80	50%
Vector search (Qdrant)	10–20	10%
Result formatting	5–10	5%
Total	115–290	100%

Table 4.2: PATH 2 latency profile (P50 to P95)

Key observations:

- Bottleneck: Image encoding (FashionSigLIP forward pass) dominates runtime
- Network: Upload speed depends on user connection; typically <200ms in real deployment
- Search: Qdrant query is negligible (<20ms for 12K-item index)
- P95 target: Keep total latency <500ms for responsive UX

4.4.2 Throughput and Scaling

With a single FastAPI worker and one shared FashionSigLIP model instance:

- Single request: 150ms end-to-end
- Concurrent requests (10): 1.5s (encoder is CPU/GPU bottleneck; requests queue)
- Recommendation: Deploy 2–3 FastAPI worker processes; consider GPU acceleration for scale

4.5 Integration with Flutter Frontend

The Flutter web UI integrates PATH 2 through a dedicated image upload button:

```
class ChatScreen extends ConsumerWidget {
  Future<void> _pickAndSearchImage(BuildContext context, WidgetRef
   ref) async {
    // File picker dialog
    final result = await FilePicker.platform.pickFiles(
      type: FileType.image,
      allowedExtensions: ['png'], // PATH 2 specifically enforces
   PNG
```

```
    );

    if (result != null && result.files.isNotEmpty) {
      final file = result.files.first;
      final bytes = file.bytes;

      // Show warning if file is large
      if (bytes != null && bytes.length > 3 * 1024 * 1024) {
        _warnIfLargeImage(context, bytes.length);
      }

      // Call chat provider (which triggers API)
      await ref.read(chatProvider.notifier).searchByImage(bytes);
    }
  }
}
```

Listing 4.3: Flutter Image Search Integration — chat_screen.dart

Flutter workflow:

1. User clicks image icon next to send button
2. File picker opens (filters .png)
3. User selects image; bytes loaded into memory
4. If size >3MB, friendly warning displayed (but upload proceeds)
5. Chat provider sends POST to `/api/path2/image-search`
6. Results displayed in chat bubbles with product thumbnails
7. Uploaded image also rendered inline in user message bubble

4.5.1 Image Rendering in Chat Bubbles

The chat UI now renders uploaded images as thumbnails (200x200px) directly in the user message bubble:

```
class _UserBubble extends StatelessWidget {
  final ChatMessage message;

  @override
  Widget build(BuildContext context) {
    return Align(
      alignment: Alignment.centerRight,
```

```
      child: Container(
        constraints: BoxConstraints(maxWidth: 300),
        padding: EdgeInsets.all(12),
        decoration: BoxDecoration(
          color: Colors.blue.shade100,
          borderRadius: BorderRadius.circular(12),
        ),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.end,
          children: [
            // Text content
            if (message.text.isNotEmpty)
              Text(message.text, style: TextStyle(fontSize: 14)),

            // Image thumbnail (if present)
            if (message.imageBytes != null)
              Padding(
                padding: EdgeInsets.only(top: 8),
                child: ClipRRect(
                  borderRadius: BorderRadius.circular(8),
                  child: Image.memory(
                    message.imageBytes!,
                    width: 200,
                    height: 200,
                    fit: BoxFit.cover,
                    errorBuilder: (context, error, stackTrace) {
                      return Container(
                        width: 200,
                        height: 200,
                        color: Colors.grey.shade300,
                        child: Column(
                          mainAxisAlignment: MainAxisAlignment.
center,
                          children: [
                            Icon(Icons.broken_image),
                            SizedBox(height: 8),
                            Text('Failed to load image'),
                          ],
                        ),
                      );
                    },
                  ),
```

```
                  ),
                ),
              ],
            ),
          ),
        );
    }
}
```

Listing 4.4: Image Display in Chat Bubbles — chat_bubble.dart

UX features:

- Inline display: Images shown immediately in chat as user sends them
- Size control: Fixed 200x200px thumbnail for clean layout
- Error handling: Graceful fallback if image bytes corrupt (shows icon + message)
- Context preservation: Image stored with ChatMessage for session history

4.6 Architectural Isolation

PATH 2 is intentionally decoupled from PATH 1 to enable independent evolution:

Component	PATH 1	PATH 2
Endpoint	`/api/chat`	`/api/path2/image-search`
Search module	`search/search_engine.py`	`search/image_query_engine.py`
Agent loop	Yes (ReAct, intent, clarification)	No (direct retrieval)
LLM synthesis	Yes (Gemini summary)	No (raw product list)
Reranking	Yes (BGE)	No (direct Qdrant results)
Feature flag	Shared `ENABLE_PATH2_IMAGE_SEARCH`	Same flag controls both endpoints
Test suite	`tests/test_search.py`	`tests/test_image_search.py`

Table 4.3: Architectural isolation between PATH 1 and PATH 2

Benefits of isolation:

- No PATH 1 regression risk — changes to PATH 2 image search do not affect text search
- Simpler logic — PATH 2 skips intent classification, clarification, reranking (unnecessary for image search)
- Independent scaling — can cache PATH 2 results or add dedicated GPU without affecting PATH 1 agent

- Future flexibility — easy to add PATH 2 into PATH 1 loop later (e.g., multi-modal fusion in PATH 3)

4.7 Test Coverage for PATH 2

```python
def test_path2_disabled_returns_503(monkeypatch):
    monkeypatch.setenv("ENABLE_PATH2_IMAGE_SEARCH", "false")
    client = TestClient(api_main.app)

    resp = client.post(
        "/api/path2/image-search",
        files={"image": ("query.png", _tiny_png_bytes(), "image/png")},
        data={"session_id": "s1", "top_k": "3"},
    )
    assert resp.status_code == 503
    assert "disabled" in resp.text.lower()

def test_path2_rejects_non_png(monkeypatch):
    monkeypatch.setenv("ENABLE_PATH2_IMAGE_SEARCH", "true")
    client = TestClient(api_main.app)

    resp = client.post(
        "/api/path2/image-search",
        files={"image": ("query.jpg", b"not-png", "image/jpeg")},
        data={"session_id": "s1", "top_k": "3"},
    )
    assert resp.status_code == 415
    assert ".png" in resp.text.lower()

def test_path2_success_returns_products(monkeypatch):
    monkeypatch.setenv("ENABLE_PATH2_IMAGE_SEARCH", "true")
    # Mock search execution to decouple from vector DB connection
    monkeypatch.setattr(
        api_main,
        "_run_path2_image_search",
        lambda raw, top_k: [{"image_id": "img-1", "score": 0.99}][:top_k],
    )
    client = TestClient(api_main.app)

    resp = client.post(
```

```
        "/api/path2/image-search",
        files={"image": ("query.png", _tiny_png_bytes(), "image/png")},
        data={"session_id": "s1", "top_k": "1"},
    )
    assert resp.status_code == 200
    body = resp.json()
    assert body["mode"] == "path2"
    assert body["count"] == 1
    assert body["products"][0]["image_id"] == "img-1"

def test_path1_contract_unchanged_with_path2_toggle(monkeypatch):
    """Ensures PATH 1 chat streaming remains fully operational regardless of PATH 2 flag"""
    client = TestClient(api_main.app)

    monkeypatch.setenv("ENABLE_PATH2_IMAGE_SEARCH", "false")
    resp_disabled = client.post("/api/chat/stream", json={})
    assert resp_disabled.status_code == 422 # Standard validation error, not 503

    monkeypatch.setenv("ENABLE_PATH2_IMAGE_SEARCH", "true")
    resp_enabled = client.post("/api/chat/stream", json={})
    assert resp_enabled.status_code == 422
```

Listing 4.5: PATH 2 Test Suite — tests/test_path2_image_search.py

Test coverage summary:

- Configuration state: Endpoint properly respects `ENABLE_PATH2_IMAGE_SEARCH` (returns 503 if disabled).
- Validation: Strict file type enforcement rejecting non-PNG uploads (returns 415).
- Success Flow: Successfully processes requests, integrates with `top_k` constraints, and formats `path2` responses.
- Isolation Validation: Ensures toggling the PATH 2 feature flag has no detrimental effect on the core PATH 1 streaming endpoints.

All 4 tests passing: Configuration state, Strict validation, Search success, Path isolation.

4.8 Scenario: Finding an Outfit for a Company Meeting

4.8.1 Turn 1 — Initial Query

Turn 1 Walkthrough

User: "I want to find an outfit for a company meeting"
[Step 1 — Intent Classification]

- Classified intent: `outfit_request`
- Confidence: $0.72 \geq 0.6 \rightarrow$ sufficient to proceed
- Extracted filters: {occasion: "office"}

[Step 2 — Memory Lookup (PostgreSQL)]

- New session $\rightarrow$ no stored preferences

[Step 3 — Orchestrator Planning]
Gemini creates plan: `[{``tool'': ``outfit_hints'', ``occasion'': ``office''}, {``tool'': ``search'', ``query'': ``formal shirt office meeting''}]`
[ReAct Loop — Iterations 1–2]
Action: `outfit_hints(occasion=``office'')` $\rightarrow$ categories = [Shirt, Pants, Shoes]
Action: `search(``formal shirt office meeting'')` $\rightarrow$ 8 results after RRF + reranker
[LLM Synthesis — Gemini]
Agent responds: "For a company meeting, I suggest a smart-casual outfit: (1) A slim-fit cotton dress shirt — elegant and professional; (2) Grey or navy chino trousers; (3) Clean leather or loafer shoes."

4.8.2 Turn 2 — Follow-up with Additional Details

Turn 2 Walkthrough

User: "Give me a white shirt and navy trousers"
[Step 1]: Intent: `follow_up`, Confidence: 0.88
Filters: {category: "Shirt", color: "White"}
[Step 2]: Memory Agent loads session context: office occasion
[ReAct Loop]:
BM25 exact match: "Shirt White" → highest rank
FashionSigLIP semantic match: "slim fit woven cotton formal shirt"
[Result]:
Agent responds + 6-image gallery: "Here are my suggestions: (1) White slim-fit cotton shirt [SP1, SP2, SP3] (2) Navy blue chino trousers [SP4, SP5]. The white + navy combination is a classic for corporate environments."

4.8.3 Refusal Scenario — Product Not in Database

Handling No Results Found

User: "Find me a Gucci shirt"
[Intent]: `text_search`, confidence 0.91
[ReAct Loop]: search "Gucci luxury fashion" → 0 results with relevance ≥ 0.3
[Fallback]: Lower threshold → still not sufficient
Agent politely refuses:
"Sorry, the current collection does not include Gucci brand products. However, I can find items with a similar style (luxury, minimalist, formal). Would you like me to try that instead?"

Chapter 5
IMPLEMENTATION

5.1 Motivation for Migrating from CSV to PostgreSQL

> **Note**
>
> CSV is unsuitable for server environments because: (1) it does not support concurrent access, (2) data is lost when a container restarts, and (3) conditional queries are inefficient. PostgreSQL resolves all of these issues while providing full ACID compliance.

5.2 Database Schema

```
-- Main product table
CREATE TABLE IF NOT EXISTS fashion_items (
    id              SERIAL PRIMARY KEY,
    image_id        TEXT UNIQUE NOT NULL,
    label           TEXT,
    color           TEXT,
    caption         TEXT,
    source          TEXT DEFAULT 'kaggle',
    image_path      TEXT,
    created_at      TIMESTAMPTZ DEFAULT NOW()
);

-- User session table
CREATE TABLE IF NOT EXISTS sessions (
    session_id          TEXT PRIMARY KEY,
    created_at          TIMESTAMPTZ DEFAULT NOW(),
    query_history       JSONB DEFAULT '[]',
    liked_items         JSONB DEFAULT '[]',
    gender              TEXT DEFAULT 'unisex',
    gender_hint_enabled BOOLEAN DEFAULT TRUE,
    year_of_birth       INTEGER
);

-- Conversation message table
```

```sql
CREATE TABLE IF NOT EXISTS session_messages (
    id              SERIAL PRIMARY KEY,
    session_id      TEXT REFERENCES sessions(session_id),
    role            TEXT NOT NULL,  -- 'user' or 'assistant'
    content         TEXT NOT NULL,
    created_at      TIMESTAMPTZ DEFAULT NOW()
);
```

Listing 5.1: PostgreSQL schema for the Fashion Agent

5.3 Data Access Layer

```python
import psycopg2

def _get_conn():
    """Connect to PostgreSQL from environment variables."""
    return psycopg2.connect(
        host=os.getenv("PGHOST", "localhost"),
        port=int(os.getenv("PGPORT", "5432")),
        dbname=os.getenv("PGDATABASE", "fashion_rag"),
        user=os.getenv("PGUSER", "fashion_user"),
        password=os.getenv("PGPASSWORD", ""),
    )

def create_session() -> str:
    """Create a new session and return its UUID."""
    sid = str(uuid.uuid4())
    with _get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute(
                "INSERT INTO sessions (session_id) VALUES (%s)",
                (sid,))
        conn.commit()
    return sid

def get_preferences(session_id: str) -> dict:
    """Extract accumulated preferences from query history."""
    with _get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute(
                "SELECT query_history FROM sessions "
                "WHERE session_id = %s", (session_id,))
```

```
            row = cur.fetchone()
    if not row or not row[0]:
        return {}
    # Analyze filters from history to derive preferred colors/
    categories
    history = row[0] if isinstance(row[0], list) else []
    colors, categories = Counter(), Counter()
    for entry in history:
        filters = entry.get("filters", {})
        if filters.get("color"):
            colors[filters["color"]] += 1
        if filters.get("category"):
            categories[filters["category"]] += 1
    return {
        "preferred_colors": [c for c, _ in colors.most_common(3)],
        "preferred_categories":
            [c for c, _ in categories.most_common(3)],
    }
```

Listing 5.2: PostgreSQL Data Access Layer — agent/memory.py

Task breakdown:

- Environment-based connection: Connects to PostgreSQL via environment variables, suitable for Docker deployment.
- JSONB: Stores liked items and query history flexibly without requiring additional tables.
- Preference extraction: Automatically aggregates the top-3 colors and categories from query history.

5.4 Gradio Interface

```
def create_gradio_app(app):
    """Create Gradio UI integrated with FastAPI."""
    with gr.Blocks(
        theme=gr.themes.Monochrome(),
        title="Fashion Agent v2.0"
    ) as demo:
        gr.Markdown("## Fashion Agent v2.0")
        session_state = gr.State({"session_id": None})
        chatbot = gr.Chatbot(label="Fashion Assistant",
                             height=500)
```

```
        with gr.Row():
            msg = gr.Textbox(
                placeholder="Find a blue shirt for a meeting...",
                label="Your question", scale=5)
            send = gr.Button("Send", variant="primary")

        gallery = gr.Gallery(label="Products found",
                             columns=3, height=300)
        styling_box = gr.Textbox(label="Outfit suggestions")

        gr.Examples(inputs=msg, examples=[
            ["Find an outfit for a company meeting"],
            ["White shirt, navy trousers"],
            ["red dress suitable for party"],
        ])
    return demo
```

Listing 5.3: Gradio Chat Interface — api/main.py

5.5 Docker Deployment Configuration

```
# docker-compose.yml
services:
  postgres:       # PostgreSQL 16 Alpine (:5432)
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: fashion_rag
      POSTGRES_USER: fashion_user

  qdrant:         # Qdrant Vector DB (:6333)
    image: qdrant/qdrant:latest

  fashion-api:    # FastAPI + Gradio (:8000)
    build: .
    environment:
      PGHOST: postgres
      QDRANT_HOST: qdrant
      GEMINI_API_KEY: ${GEMINI_API_KEY}
    volumes:
      - ./models:/app/models    # HuggingFace cache
      - ./images:/app/images:ro # Product images
    depends_on:
```

```
      postgres: {condition: service_healthy}
      qdrant:   {condition: service_healthy}

  cloudflared:    # Cloudflare Tunnel (public URL)
    image: cloudflare/cloudflared:latest
    command: tunnel run
```

Listing 5.4: Docker Compose — 4 services

```
# PostgreSQL
PGDATABASE=fashion_rag
PGUSER=fashion_user
PG_PASSWORD=your_secure_password

# API Keys
GEMINI_API_KEY=your_gemini_api_key
QDRANT_API_KEY=optional_qdrant_key
CF_TUNNEL_TOKEN=cloudflare_tunnel_token

# Paths
DATASET_IMAGES_HOST_PATH=./images
```

Listing 5.5: Environment variables (.env)

Chapter 6

RESULT

6.1 Evaluation Metrics

Metric	Meaning	RAG v1.0	Agent v2.0
Recall@5 (PATH 1)	Top-5 correct category/color	$\geq$85%	$\geq$88%
MRR	Position of first correct result	$\geq$0.75	$\geq$0.80
Hit Rate@5	$\geq$1 correct result in top-5	$\geq$90%	$\geq$93%
Task Completion	No crash, returns results	N/A	$\geq$95%
Faithfulness	No hallucinated information	N/A	$\geq$0.87
Latency P95	Total ReAct loop time	$<$15s	$<$15s

Table 6.1: System evaluation metrics comparing RAG v1.0 and Agent v2.0

6.2 Data Collection and Evidence Integrity

To make the final conclusions auditable, this thesis reports not only model metrics but also the provenance of collected data, aggregation boundaries, and data-quality checks. The evidence pipeline is based on operational PostgreSQL logs generated by the deployed system.

6.2.1 Canonical Data Sources

Evidence Signal	Source Table / View	Purpose in Evaluation
LLM token usage	`llm_token_usage`, `session_token_summary`, `mode_cost_summary`	Measure input/output token consumption per session and estimate operational cost by orchestration mode.
Behavior funnel	`product_impressions`, `product_clicks`, `selected_items`, `product_intents`, `user_orders`, `session_path_funnel_summary`	Quantify user progression from impression to selection (cart add), purchase intent, and order conversion.
Retrieval quality	Evaluation query set + top-k ranked outputs	Compute Recall@5, MRR, Hit Rate@5, and faithfulness-based quality indicators.

Table 6.2: Canonical evidence sources used for thesis conclusions

6.2.2 Metric Definitions and Cohort Boundaries

Token metrics. For a cohort of sessions $\mathscr{S}$ with valid synthesis records (`call_name='synthesis`
token usage is aggregated as:

$$T_{in} = \sum_{s \in \mathscr{S}} \text{input\_tokens}_s, \quad T_{out} = \sum_{s \in \mathscr{S}} \text{output\_tokens}_s, \quad T_{total} = T_{in} + T_{out} \tag{6.1}$$

When orchestration is enabled, orchestrator token fields are reported separately through `mode_cost_summary` to prevent hidden cost inflation.

Add-to-cart and behavior metrics. In this system, a cart-add event corresponds to a row in `selected_items`. Let I = impressions, C = clicks, A = cart-add selections, W = "will_buy" intent, O = orders:

$$\text{CartAddRate} = \frac{A}{I}, \quad \text{WillBuyRate} = \frac{W}{A}, \quad \text{ConversionRate} = \frac{O}{|\mathscr{S}|} \tag{6.2}$$

These definitions align with analytics endpoints in `api/analytics.py` and the funnel view in `agent/memory.py`.

Accuracy metrics. Retrieval quality is reported using Recall@5, MRR, and Hit Rate@5 (Table 6.3), computed on an evaluation cohort of N_{query} labeled queries. Every reported score must include N_{query} to avoid over-interpreting small samples.

Metric	Definition	Report with
Recall@5	Proportion of queries where at least one relevant item appears in top-5 results.	N_{query}
MRR	Mean reciprocal rank of the first relevant retrieved item.	N_{query}
Hit Rate@5	Fraction of queries with $\geq$1 relevant result in top-5.	N_{query}

Table 6.3: Accuracy reporting template with mandatory sample-size context

6.2.3 Inclusion, Exclusion, and Time-Window Rules

To ensure reproducibility, all aggregates in this chapter SHALL follow the same boundary policy:

1. Include only sessions inside the declared collection window $[t_{start}, t_{end}]$.

2. Exclude sessions with missing key fields for the target metric (e.g., missing token fields for token analysis).

3. Exclude duplicate interaction rows when the same session-event key appears multiple times due to retries.

4. Report both the original cohort size and the post-filter size to make exclusion impact explicit.

6.2.4 Data Quality and Validity Checks

Check	Validation Rule	Interpretation if Failed
Missingness	Required fields are non-null (session_id, event timestamp, metric-specific columns).	Report as undercount risk; do not claim strong support from affected metric.
Consistency	Event semantics are stable (e.g., cart-add is always `selected_items`).	Re-map metric definitions before comparing across sections.
Duplicate events	No duplicated session-event records after deduplication policy.	Treat inflated rates as invalid until corrected.
Sample balance	Cohort not dominated by one subgroup/mode.	Downgrade claim strength to "partially supported" or "inconclusive."

Table 6.4: Validity checks for deciding whether collected data is trustworthy

This validity layer is the decision gate for "data right or wrong": metrics are considered trustworthy only when they pass the checks in Table 6.4; otherwise, conclusions must be weakened accordingly.

Chapter 7
DISCUSSION

7.1 Design Strengths

1. Intent-first: The agent classifies intent before acting — prevents incorrect searches.
2. Clarification gate: For ambiguous queries, asks instead of guessing — improves quality.
3. Graceful degradation: Politely refuses when no product is found, with alternative suggestions.
4. Hybrid retrieval: BM25 + FashionSigLIP + RRF leverages the strengths of both approaches.
5. PostgreSQL: Persistent, scalable, supports concurrent access better than CSV.
6. Semantic token (caption): LLM-generated descriptions enable deeper embedding understanding beyond short labels.
7. Query Expansion: Gemini-powered synonym queries increase recall for short queries.
8. Cross-encoder Reranker: BGE v2-m3 reranks results for higher precision.

7.2 Risks and Mitigation

Risk	Severity	Mitigation
High latency (multiple Gemini calls)	High	Parallel tool calls, cached expansion
LLM hallucination	High	RAGAS faithfulness $\geq$0.87, fallback templates
BM25 rebuild on restart	Low	Serialize BM25 index to disk
PostgreSQL cold start	Low	Connection pooling
Gemini API rate limit	Medium	Batch processing, exponential backoff

Table 7.1: Risk matrix and corresponding mitigation strategies

Chapter 8

CONCLUSION

8.1 Conclusion

This project successfully built an intelligent multimodal fashion search system following a clear two-phase roadmap. The RAG v1.0 phase established a hybrid search foundation with FashionCLIP 2.0 and BM25. Agent v2.0 then expanded into two paths: PATH 1 upgraded to a proactive architecture featuring a ReAct loop, intent classification, a clarification gate, and persistent memory via PostgreSQL, while PATH 2 added image-to-image composed search for visual product retrieval.

Version v2.0 represents significant improvements over v1.0:

- Embedding: Upgraded from FashionCLIP (512-d) to Marqo-FashionSigLIP (768-d).
- Query Expansion: Added Gemini-powered synonym expansion for short queries.
- Reranker: Integrated BGE Reranker v2-m3 cross-encoder for improved precision.
- Deployment: Docker Compose stack with 4 services (PostgreSQL + Qdrant + FastAPI + Cloudflare Tunnel).

8.1.1 Evidence-Based Claim Mapping for Final Conclusion

Final claims are mapped to evidence strength so that conclusions remain proportional to verified data:

Claim	Primary Evidence	Status
Agent v2.0 improves retrieval quality over v1.0.	Recall@5, MRR, Hit Rate@5 (Section 6.2 and Section 6.2.2).	Supported
Agent behavior is robust in multi-turn interaction.	Task completion, faithfulness, and clarification behavior logs.	Supported
Operational LLM cost is explainable and monitorable.	`llm_token_usage`, `session_token_summary`, mode cost aggregation.	Partially supported
User purchase tendency is improved by recommendations.	CartAddRate / WillBuyRate / ConversionRate from behavior funnel.	Inconclusive (depends on cohort size and balance)

Table 8.1: Claim-to-evidence mapping used in the final thesis conclusion

Therefore, the final conclusion follows three rules: (1) only claims marked Supported are stated as confirmed findings, (2) Partially supported claims are reported with operational caveats, and (3) Inconclusive claims are retained as hypotheses for future data collection.

Current evidence limitations.

- Token-to-cost estimates depend on fixed public list prices and may vary over time.
- Behavior metrics are sensitive to sample imbalance across orchestration modes and user groups.
- Conversion-style claims require larger longitudinal cohorts than early-stage thesis collection windows.

Future development directions:

- Phase 3: Scale dataset to 15K–100K products, fine-tune the reranker, Redis session cache.
- Phase 4: Virtual Try-on (IDM-VTON), fine-tune FashionSigLIP on proprietary data, A/B testing.
- PATH 2: Image-to-Image search with Composed Search.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474, 2020.

[2] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in Proceedings of the International Conference on Learning Representations (ICLR), 2023.

[3] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in Proceedings of the 38th International Conference on Machine Learning (ICML), pp. 8748–8763, 2021.

[4] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, "Sigmoid loss for language image pre-training," in Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 11975–11986, 2023.

[5] Marqo, "Marqo-FashionSigLIP: A SigLIP-based fashion embedding model." Hugging Face. `https://huggingface.co/Marqo/marqo-fashionSigLIP`, 2024.

[6] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.

[7] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 758–759, 2009.

[8] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-Pack: Packed resources for general Chinese embeddings," in Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 641–649, 2024.

[9] Google DeepMind, "Gemini 2.5 Flash: Multimodal large language model." Technical report. Google. `https://ai.google.dev/gemini-api/docs`, 2024.

[10] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," in Proceedings of the 18th Conference of

the European Chapter of the Association for Computational Linguistics: System Demonstrations, pp. 150–158, 2024.

[11] Qdrant, "Qdrant: Vector similarity search engine." Documentation. https://qdrant.tech/documentation/, 2024.

[12] The PostgreSQL Global Development Group, "PostgreSQL 16 documentation." https://www.postgresql.org/docs/16/, 2024.

[13] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan, and J. Zou, "Gradio: Hassle-free sharing and testing of ML models in the wild," arXiv preprint arXiv:1906.02569, 2019.

[14] Kaggle Community, "Clothing dataset full." Data set. Kaggle. https://www.kaggle.com/datasets/agrigorev/clothing-dataset-full, 2023.

Appendix A
LISTINGS

A.1 Supplementary Equations and Derivations

This appendix contains supplementary mathematical material referenced in the main text.

A.2 Cosine Similarity from L2-Normalized Vectors

For two vectors $u, v \in \mathbb{R}^d$, the cosine similarity is:

$$\cos(u, v) \;=\; \frac{u \cdot v}{\|u\|_2 \, \|v\|_2} \tag{A.1}$$

When both vectors are L2-normalized (i.e. $\|u\|_2 = \|v\|_2 = 1$), Equation (A.1) reduces to the dot product:

$$\cos(\hat{u}, \hat{v}) \;=\; \hat{u} \cdot \hat{v} \tag{A.2}$$

This is why `FashionEmbedder` normalizes embeddings before storing them in Qdrant — it allows the use of fast dot-product search rather than full cosine computation.

A.3 BM25 Scoring Function

The BM25 score of a document d for a query $q = \{q_1, q_2, \ldots, q_n\}$ is:

$$\mathrm{BM25}(d, q) \;=\; \sum_{i=1}^{n} \mathrm{IDF}(q_i) \cdot \frac{f(q_i, d) \cdot (k_1 + 1)}{f(q_i, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\mathrm{avgdl}}\right)} \tag{A.3}$$

where $f(q_i, d)$ is the term frequency of q_i in d, $|d|$ is the document length, avgdl is the average document length in the corpus, and k_1, b are free parameters (typically $k_1 \in [1.2, 2.0]$ and $b = 0.75$).

A.4 Sample Configuration Files

A.5 PostgreSQL Initialization Script

This appendix contains a sample SQL initialization script that may be used to bootstrap a fresh PostgreSQL instance for the Fashion Agent. See Table A.1 for the required environment variables.

Variable	Description
PGHOST	PostgreSQL host (e.g., postgres in Docker network)
PGPORT	PostgreSQL port (default: 5432)
PGDATABASE	Database name (fashion_rag)
PGUSER	Database user (fashion_user)
PGPASSWORD	Database password
GEMINI_API_KEY	Google Gemini API key
QDRANT_HOST	Qdrant host (e.g., qdrant)

Table A.1: Required environment variables for deployment